

**Instituto Tecnológico CTC Colonia**

**Bases de Datos**

**II**

# **Obligatorio**

**Andrés Klett**

**Paula Camacho**

**Cristhian Castro**

**2026**

# Índice

|                               |    |
|-------------------------------|----|
| Índice                        | 2  |
| Declaración de Autoría        | 3  |
| MER (Modelo Entidad Relación) | 4  |
| Justificación del Diseño      | 5  |
| Creación de Tablas            | 7  |
| Índices                       | 12 |
| Consultas                     | 13 |
| Procedimientos y Funciones    | 16 |
| Disparadores (Triggers)       | 19 |

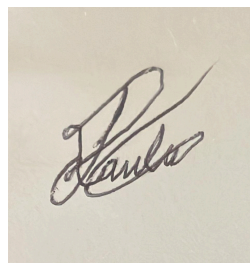
## Declaración de Autoría

Nosotros, Cristhian Castro y Paula Camacho, declaramos que el trabajo que se presenta es de nuestra propia mano. Puedo asegurar que:

- La obra fue producida mientras cursamos el tercer semestre de analista programador en la materia Bases de datos II con el docente Andrés Klett;
- Hemos tenido en cuenta las clases dictadas por el docente Andrés Klett, quien además proporcionó el material;
- Cuando hemos consultado el trabajo publicado por otros, lo hemos atribuido con claridad;
- Cuando hemos citado obras de otros, hemos indicado las fuentes. Con excepción de estas citas, la obra es enteramente nuestra;
- Cuando la obra se basa en trabajo realizado conjuntamente con otros, hemos explicado claramente que fue construido por otros, y qué fue contribuido por nosotros;
- Ninguna parte de este trabajo ha sido publicada previamente a su entrega, excepto donde se han realizado las aclaraciones correspondientes.

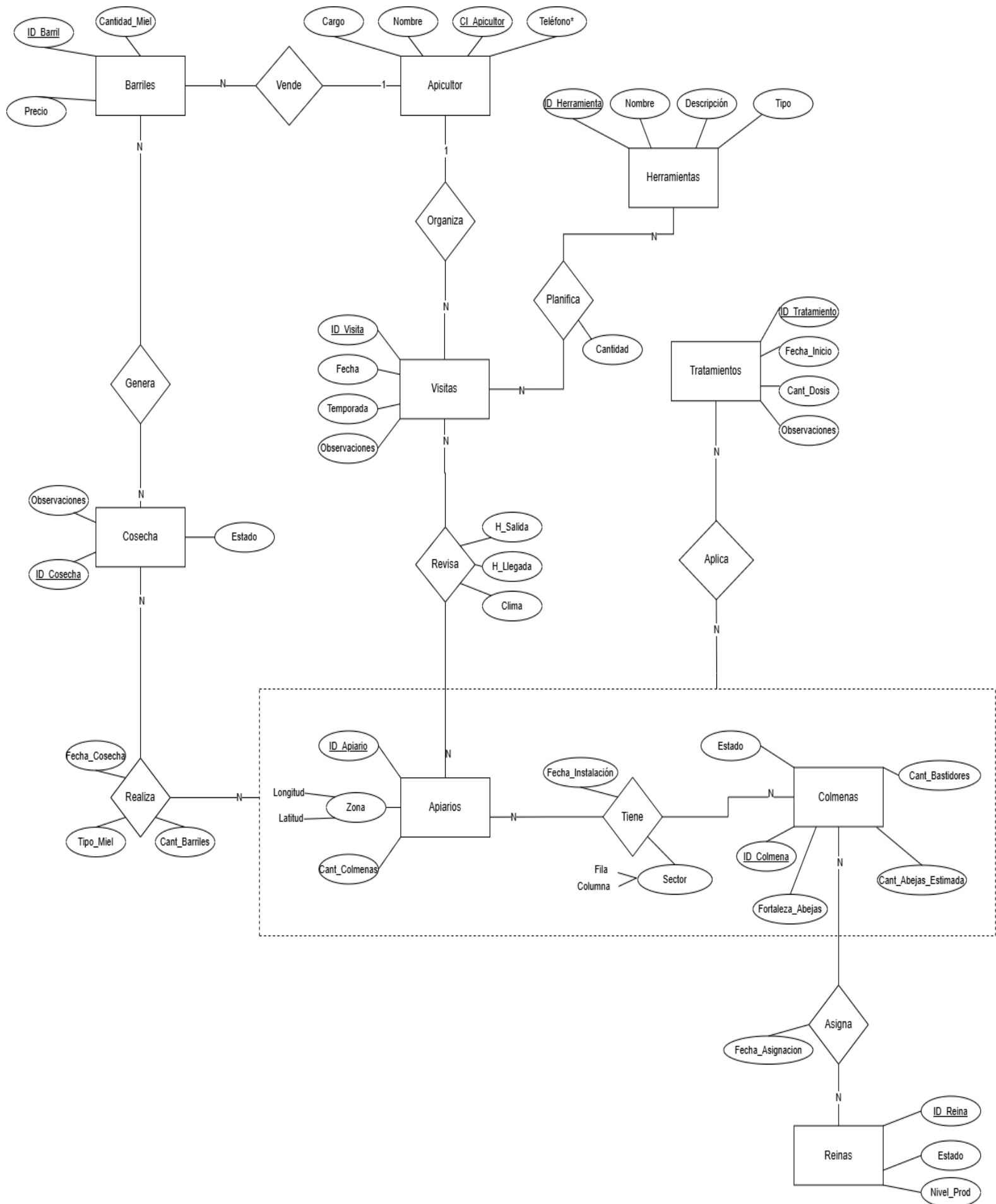


Cristhian Castro



Paula Camacho

## MER (Modelo Entidad Relación)



## Justificación del Diseño

Las decisiones tomadas del diseño para la arquitectura están enfocadas en dos pilares fundamentales: la escalabilidad del sistema y la capacidad de llevar un historial detallado de las operaciones de Zánganos S.A. Como el software debe acompañar el crecimiento de la empresa. La estructura se pensó para que pueda adaptarse sin problemas a la futura incorporación de empleados y el aumento del volumen de la producción de miel. También, se prioriza el registro cronológico de cada evento en el campo, transformando los datos diarios en un archivo histórico valioso.

la entidad **Apicultor** lleva el atributo *Cargo* porque, aunque hoy Matías hace todo solo, el día de mañana puede contratar empleados; de esta forma, el software permite que solo Matías (como Administrador) pueda manejar la venta de **Barriles**, separando su rol del de los futuros peones de campo. Por eso también su relación de 1 a N.

Por otro lado, para que no se pierda el rastro de los movimientos en el monte de Rivera, se armó una relación de muchos a muchos entre **Apiarios** y **Colmenas**. Esto se hizo así para que funcione como un historial: cuando una colmena se mueve de un apiario a otro por temas de floración o clima, el sistema no borra el pasado, sino que guarda todo el camino que hizo esa colmena. Así, Matías puede mirar hacia atrás y entender por qué rindió más o menos según el lugar donde estuvo instalada y detectar enfermedades o pesticidas. La relación que hay entre estas entidades **Tiene** el atributo de la *Fecha\_Instalacion* junto con la ubicación (*Fila, Columna y Sector*) ayudan a trazar el historial de forma correcta.

El diseño incluye restricciones específicas que aseguran la buena inserción de datos y que sean intuitivos para Matías. En la tabla **Colmenas**, el campo *Fortaleza\_Abejas* está limitado únicamente a los valores 'Fuerte', 'Media' o 'Débil', haciendo un estándar para el criterio de evaluación de las abejas.

También, en la tabla **Planifica** el sistema exige ingresar de forma obligatoria la cantidad de herramientas y bastidores para cada visita; esto fuerza una preparación previa y soluciona el problema de llegar al apiario sin el equipamiento necesario.

Siguiendo la misma lógica de control histórico, la relación entre **Colmenas** y **Reinas** también se diseñó como una relación de muchos a muchos N a N a través de la tabla **Asigna**, incorporando el atributo *Fecha\_Asignacion*. En la práctica de la apicultura, las reinas se cambian cuando envejecen, bajan su producción o por una enjambrazón. Al registrar el día y la hora exacta en que una reina entra a una colmena, el sistema no borra el pasado cuando Matías introduce una reina nueva, sino que guarda la línea de tiempo completa. Esto resuelve el problema de la falta de registros históricos, permitiendo a Matías analizar el rendimiento del panal hacia atrás y descubrir qué reinas o qué genéticas dieron los mejores resultados en cada cosecha.

Por último, al requerir la fecha exacta en la tabla **Aplica**, la base de datos registra el momento exacto en que se inicia un tratamiento sanitario, permitiendo controlar con precisión el vencimiento y la aplicación de las segundas dosis para proteger la salud de las colmenas.

## Tablas normalizadas

Apicultor (CI\_Apicultor, Nombre, Cargo)

Apicultor\_Telefono (CI\_Apicultor , Teléfono) → multivaluado

Herramientas (ID\_Herramienta, Nombre, Descripción, Tipo)

Planifica (ID\_Visita, ID\_Herramienta, Cantidad)

Visitas (ID\_Visita, Fecha, Temporada, Observaciones, CI\_Apicultor)

Realiza (ID\_Cosecha, ID\_Apiario, Fecha\_Cosecha, Tipo\_Miel, Cant\_Barriles)

Cosecha (ID\_Cosecha, Observaciones, Estado)

Genera (ID\_Barril, ID\_Cosecha)

Barriles (ID\_Barril, Cantidad\_Miel, Precio, CI\_Apicultor)

Revisa (ID\_Visita, ID\_Apiario, H\_Salida, H\_Llegada, Clima)

Apiarios (ID\_Apiario, Ubicación, Longitud, Latitud, Zona, Cant\_Colmenas)

Tiene (ID\_Apiario, ID\_Colmena, Fecha\_Instalación, Sector, Fila, Columna)

Colmenas (ID\_Colmena, Estado, Cant\_Bastidores, Cant\_Abejas\_Estimada, Fortaleza\_Abejas)

Asigna (ID\_Colmena , ID\_Reina, Fecha\_Asignacion)

Reinas (ID\_Reina, Estado, Nivel\_Prod)

Aplica (ID\_Colmena, ID\_Tratamiento)

Tratamientos (ID\_Tratamiento, Fecha\_Inicio, Cant\_Dosis, Observaciones)

## Creación de Tablas

Creación de tipos de datos:

```
CREATE TYPE TipoNombre FROM VARCHAR(100) NOT NULL;
```

```
CREATE TYPE TipoObservacion FROM VARCHAR(255);
```

Entidades:

```
CREATE TABLE Apicultor (
```

```
    CI_Apicultor INT PRIMARY KEY,  
    Nombre TipoNombre,  
    Cargo VARCHAR(20),
```

```
    CONSTRAINT CK_Apicultor_Cargo CHECK (Cargo IN ('Administrador de Apiario', 'Apicultor  
de Campo'))  
);
```

```
CREATE TABLE Apicultor_Telefono (
```

```
    CI_Apicultor INT,  
    Telefono INT,  
    CONSTRAINT PK_Apicultor_Telefono PRIMARY KEY (CI_Apicultor, Telefono),
```

```
    CONSTRAINT FK_ApTel_Apicultor FOREIGN KEY (CI_Apicultor)  
        REFERENCES Apicultor(CI_Apicultor)  
);
```

```
CREATE TABLE Barriles (
```

```
    ID_Barril INT IDENTITY(1,1) PRIMARY KEY,  
    Cantidad_Miel DECIMAL(10,2) NOT NULL,  
    Precio DECIMAL(10,2) NOT NULL,  
    CI_Apicultor INT,
```

```
    CONSTRAINT FK_Barriles_Apicultor FOREIGN KEY (CI_Apicultor)  
        REFERENCES Apicultor(CI_Apicultor)  
);
```

```
CREATE TABLE Herramientas (
```

```
    ID_Herramienta INT IDENTITY(1,1) PRIMARY KEY,  
    Nombre TipoNombre,  
    Descripcion VARCHAR(255),  
    Tipo VARCHAR(50)
```

```
);
```

```

CREATE TABLE Visitas (
    ID_Visita INT IDENTITY(1,1) PRIMARY KEY,
    Fecha DATE NOT NULL,
    Temporada VARCHAR(50),
    Observaciones TipoObservacion,
    CI_Apicultor INT,

    CONSTRAINT FK_Visitas_Apicultor FOREIGN KEY (CI_Apicultor)
        REFERENCES Apicultor(CI_Apicultor)
);

CREATE TABLE Cosecha (
    ID_Cosecha INT IDENTITY(1,1) PRIMARY KEY,
    Observaciones TipoObservacion,
    Estado VARCHAR(50) NOT NULL
);

CREATE TABLE Apiarios (
    ID_Apiario INT IDENTITY(1,1) PRIMARY KEY,
    Longitud DECIMAL(9,6) NOT NULL,
    Latitud DECIMAL(9,6) NOT NULL,
    Zona VARCHAR(50),
    Cant_Colmenas INT NOT NULL DEFAULT 0,
);

CREATE TABLE Colmenas (
    ID_Colmena INT IDENTITY(1,1) PRIMARY KEY ,
    Estado VARCHAR(50) NOT NULL,
    Cant_Bastidores INT,
    Cant_Abejas_Estimada INT,
    Fortaleza_Abejas VARCHAR(50),
    CONSTRAINT CHK_Fortaleza_Abejas CHECK (Fortaleza_Abejas IN ('Fuerte', 'Media',
'Débil'))
);

CREATE TABLE Tratamientos (
    ID_Tratamiento INT IDENTITY(1,1) PRIMARY KEY,
    Fecha_Inicio DATE NOT NULL,
    Cant_Dosis INT NOT NULL,
    Observaciones TipoObservacion
);

CREATE TABLE Reinas (
    ID_Reina INT IDENTITY(1,1) PRIMARY KEY,
    Estado VARCHAR(50) NOT NULL,
    Nivel_Prod VARCHAR(50),
);

```



Relaciones:

```
CREATE TABLE Planifica (  
  ID_Visita INT,  
  ID_Herramienta INT,  
  Cantidad INT,  
  
  CONSTRAINT PK_Planifica PRIMARY KEY (ID_Visita, ID_Herramienta),  
  
  CONSTRAINT FK_Planifica_Visita FOREIGN KEY (ID_Visita)  
    REFERENCES Visitas(ID_Visita),  
  
  CONSTRAINT FK_Planifica_Herramienta FOREIGN KEY (ID_Herramienta)  
    REFERENCES Herramientas(ID_Herramienta)  
);
```

```
CREATE TABLE Genera (  
  ID_Barril INT,  
  ID_Cosecha INT,  
  
  CONSTRAINT PK_Genera PRIMARY KEY (ID_Barril, ID_Cosecha),  
  
  CONSTRAINT FK_Genera_Barril FOREIGN KEY (ID_Barril)  
    REFERENCES Barriles(ID_Barril),  
  
  CONSTRAINT FK_Genera_Cosecha FOREIGN KEY (ID_Cosecha)  
    REFERENCES Cosecha(ID_Cosecha)  
);
```

```
CREATE TABLE Realiza (  
  ID_Cosecha INT,  
  ID_Apiario INT,  
  Fecha_Cosecha DATE NOT NULL,  
  Tipo_Miel VARCHAR(50),  
  Cant_Barriles INT NOT NULL,  
  
  CONSTRAINT PK_Realiza PRIMARY KEY (ID_Cosecha, ID_Apiario),  
  
  CONSTRAINT FK_Realiza_Cosecha FOREIGN KEY (ID_Cosecha)  
    REFERENCES Cosecha(ID_Cosecha),  
  
  CONSTRAINT FK_Realiza_Apiario FOREIGN KEY (ID_Apiario)  
    REFERENCES Apiarios(ID_Apiario)  
);
```

```

CREATE TABLE Revisa (
    ID_Visita INT,
    ID_Apiario INT,
    H_Salida TIME,
    H_Llegada TIME,
    Clima VARCHAR(50),

    CONSTRAINT PK_Revisa PRIMARY KEY (ID_Visita, ID_Apiario),

    CONSTRAINT FK_Revisa_Visita FOREIGN KEY (ID_Visita)
        REFERENCES Visitas(ID_Visita),

    CONSTRAINT FK_Revisa_Apiario FOREIGN KEY (ID_Apiario)
        REFERENCES Apiarios(ID_Apiario)
);

```

```

CREATE TABLE Tiene (
    ID_Apiario INT,
    ID_Colmena INT,
    Fecha_Instalacion DATE,
    Sector VARCHAR(50),
    Fila VARCHAR(10),
    Columna VARCHAR(10),

    CONSTRAINT PK_Tiene PRIMARY KEY (ID_Apiario, ID_Colmena),

    CONSTRAINT FK_Tiene_Apiario FOREIGN KEY (ID_Apiario)
        REFERENCES Apiarios(ID_Apiario),

    CONSTRAINT FK_Tiene_Colmena FOREIGN KEY (ID_Colmena)
        REFERENCES Colmenas(ID_Colmena)
);

```

```

CREATE TABLE Aplica (
    ID_Colmena INT,
    ID_Tratamiento INT,

    CONSTRAINT PK_Aplica PRIMARY KEY (ID_Colmena, ID_Tratamiento),

    CONSTRAINT FK_Aplica_Colmena FOREIGN KEY (ID_Colmena)
        REFERENCES Colmenas(ID_Colmena),

    CONSTRAINT FK_Aplica_Tratamiento FOREIGN KEY (ID_Tratamiento)
        REFERENCES Tratamientos(ID_Tratamiento)
);

```

```
CREATE TABLE Asigna (  
    ID_Colmena INT,  
    ID_Reina INT,  
    Fecha_Asignacion DATETIME,  
  
    CONSTRAINT PK_Asigna PRIMARY KEY (ID_Colmena, ID_Reina),  
  
    CONSTRAINT FK_Asigna_Colmena FOREIGN KEY (ID_Colmena)  
        REFERENCES Colmenas(ID_Colmena),  
  
    CONSTRAINT FK_Asigna_Reina FOREIGN KEY (ID_Reina)  
        REFERENCES Reinas(ID_Reina)  
);
```

## Índices

- Detectar colmenas débiles o enfermas rápidamente:  
create index Indice\_Estado\_Colmena on Colmenas (Estado);
- Ver fortaleza de abejas:  
create index Indice\_Fortaleza\_Colmena on Colmenas (Fortaleza\_Abejas);
- Índice sobre la FK de Tiene hacia Apiarios, para acelerar la búsqueda de colmenas pertenecientes a un apiario:  
create index Indice\_Tiene\_IdApiario on Tiene (ID\_Apiario);
- fecha de inicio de tratamientos, se consulta frecuentemente para detectar dosis pendientes y tratamientos activos:  
create index Indice\_FechaInicio\_Tratamiento on Tratamientos (Fecha\_Inicio);
- Índice sobre el nivel de producción de las reinas, se consulta para identificar reinas candidatas a reemplazo:  
create index Indice\_NivelProd\_Reinas on Reinas (Nivel\_Prod);

## Consultas

C1. Listado de colmenas que nunca tuvieron ningún tratamiento aplicado, mostrando su ID, estado, fortaleza y la zona del apiario al que pertenecen.

```
select c.ID_Colmena, c.Estado, c.Fortaleza_Abejas, a.Zona
from Colmenas c inner join Tiene t
on c.ID_Colmena = t.ID_Colmena
inner join Apiarios a
on t.ID_Apiario = a.ID_Apiario
left join Aplica ap
on ap.ID_Colmena = c.ID_Colmena
where ap.ID_Colmena is null;
```

C2. Listado del o los apiarios con mayor cantidad de colmenas en estado 'Débil', mostrando el ID del apiario, la zona y la cantidad de colmenas débiles.

```
select a.ID_Apiario, a.Zona, count(*) as CantColmenasDebiles
from Apiarios a inner join Tiene t
on a.ID_Apiario = t.ID_Apiario
inner join Colmenas c
on t.ID_Colmena = c.ID_Colmena
where c.Fortaleza_Abejas = 'Débil'
group by a.ID_Apiario, a.Zona
having count(*) >= all (
    select count(*)
    from Tiene t2 inner join Colmenas c2
    on t2.ID_Colmena = c2.ID_Colmena
    where c2.Fortaleza_Abejas = 'Débil'
    group by t2.ID_Apiario
);
```

C3. Listado de cosechas con más barriles generados que el promedio, mostrando el ID, estado, fecha y cantidad de barriles.

```
select c.ID_Cosecha, c.Estado, r.Fecha_Cosecha, r.Tipo_Miel, r.Cant_Barriles
from Cosecha c inner join Realiza r
on c.ID_Cosecha = r.ID_Cosecha
where r.Cant_Barriles > (
    select avg(cast(Cant_Barriles as float))
    from Realiza
)
order by r.Cant_Barriles desc;
```

C4. Actualizar el estado a 'Critica' a todas las colmenas que pertenecen a apiarios cuya cantidad de colmenas registradas supera las 50.

```
update Colmenas
set Estado = 'Critica'
where ID_Colmena in (
    select t.ID_Colmena
    from Tiene t inner join Apiarios a
    on t.ID_Apiario = a.ID_Apiario
    where a.Cant_Colmenas > 50
);
```

C5. Listado de visitas realizadas en temporada de verano, mostrando el ID de visita, la fecha, la zona del apiario visitado, el clima registrado y la duración en minutos.

```
select v.ID_Visita, v.Fecha, a.Zona, r.Clima,
       datediff(minute, r.H_Salida, r.H_Llegada) as DuracionMinutos
from Visitas v inner join Revisa r
on v.ID_Visita = r.ID_Visita
inner join Apiarios a
on r.ID_Apiario = a.ID_Apiario
where v.Temporada = 'Verano'
order by v.Fecha desc;
```

C6. Colmenas con reinas en estado 'Baja' o nivel de producción 'Bajo'

```
SELECT
    c.ID_Colmena,
    c.Estado      AS EstadoColmena,
    c.Fortaleza_Abejas,
    rei.ID_Reina,
    rei.Estado    AS EstadoReina,
    rei.Nivel_Prod,
    a.ID_Apiario,
    a.Zona
FROM Colmenas c
    INNER JOIN Asigna asg
        ON c.ID_Colmena = asg.ID_Colmena
    INNER JOIN Reinass rei
        ON asg.ID_Reina = rei.ID_Reina
    INNER JOIN Tiene t
        ON c.ID_Colmena = t.ID_Colmena
    INNER JOIN Apiarios a
        ON t.ID_Apiario = a.ID_Apiario
WHERE rei.Estado = 'Baja'
    OR rei.Nivel_Prod = 'Bajo'
ORDER BY a.Zona, c.ID_Colmena;
```

C7. Eliminar los tratamientos que no fueron aplicados a ninguna colmena y cuya fecha de inicio es anterior a enero de 2025.

```
delete from Tratamientos
where Fecha_Inicio < '2025-01-01'
and ID_Tratamiento not in (
    select ID_Tratamiento
    from Aplica
);
```

C8. Aumentar en 10 la cantidad de bastidores de todas las colmenas que pertenecen a apiarios que tuvieron al menos una cosecha de tipo 'Monofloral' en los últimos 12 meses.

```
update Colmenas
set Cant_Bastidores = Cant_Bastidores + 10
where ID_Colmena in (
    select t.ID_Colmena
    from Tiene t inner join Realiza r
    on t.ID_Apiario = r.ID_Apiario
    where r.Tipo_Miel = 'Monofloral'
    and r.Fecha_Cosecha >= dateadd(month, -12, getdate())
);
```

C9. Listado de herramientas que fueron planificadas en más de una visita, mostrando su ID, nombre, tipo y cuántas veces fueron usadas.

```
select h.ID_Herramienta, h.Nombre, h.Tipo, count(p.ID_Visita) as VecesUsada
from Herramientas h inner join Planifica p
on h.ID_Herramienta = p.ID_Herramienta
group by h.ID_Herramienta, h.Nombre, h.Tipo
having count(p.ID_Visita) > 1
order by VecesUsada;
```

C10. Listado de los 3 apiarios con mayor producción total de miel en toda su historia, mostrando el ID, zona, cantidad de cosechas y total de kg.

```
select top 3 a.ID_Apiario, a.Zona,
    count(distinct r.ID_Cosecha) as CantCosechas,
    sum(b.Cantidad_Miel) as TotalMielKg
from Apiarios a inner join Realiza r
on a.ID_Apiario = r.ID_Apiario
inner join Genera g
on r.ID_Cosecha = g.ID_Cosecha
inner join Barriles b
on g.ID_Barril = b.ID_Barril
group by a.ID_Apiario, a.Zona
order by TotalMielKg desc;
```

## Procedimientos y Funciones

### 1. Procedimiento para registrar un nuevo Apiario

```
CREATE PROCEDURE sp_RegistrarApiario
@Longitud DECIMAL(9,6),
@Latitud DECIMAL(9,6),
@Zona VARCHAR(50)
AS
BEGIN
INSERT INTO Apiarios (Longitud, Latitud, Zona, Cant_Colmenas)
VALUES (@Longitud, @Latitud, @Zona, 0);
END;
```

### 2. Procedimiento para registrar una nueva cosecha y su detalle por colmena

```
CREATE PROCEDURE sp_RegistrarCosechaConDetalle
@Apiariold INT,
@Fecha DATETIME,
@Notas NVARCHAR(255),
@Colmenald INT,
@CantidadKg DECIMAL(18,2)
AS
BEGIN
BEGIN TRY
BEGIN TRANSACTION;

DECLARE @Cosechald INT;
INSERT INTO Cosechas (Apiariold, Fecha, Notas)
VALUES (@Apiariold, @Fecha, @Notas);
SET @Cosechald = SCOPE_IDENTITY();
INSERT INTO CosechasColmenas (Cosechald, Colmenald, CantidadKg)
VALUES (@Cosechald, @Colmenald, @CantidadKg);

COMMIT TRANSACTION;
END TRY
BEGIN CATCH
ROLLBACK TRANSACTION;
THROW;
END CATCH
END;
```



3. Procedimiento para cambiar el estado de una colmena

```
CREATE PROCEDURE sp_ActualizarEstadoColmena
```

```
@ID_Colmena INT,
```

```
@NuevoEstado VARCHAR(50)
```

```
AS
```

```
BEGIN
```

```
UPDATE Colmenas
```

```
SET Estado = @NuevoEstado
```

```
WHERE ID_Colmena = @ID_Colmena;
```

```
END;
```

4. Procedimiento para aplicar tratamiento a colmena

```
CREATE PROCEDURE sp_AplicarTratamientoColmena
```

```
@ID_Colmena INT,
```

```
@ID_Tratamiento INT
```

```
AS
```

```
BEGIN
```

```
IF NOT EXISTS (SELECT 1 FROM Aplica WHERE ID_Colmena = @ID_Colmena AND  
ID_Tratamiento = @ID_Tratamiento)
```

```
BEGIN
```

```
INSERT INTO Aplica (ID_Colmena, ID_Tratamiento)
```

```
VALUES (@ID_Colmena, @ID_Tratamiento);
```

```
END
```

```
ELSE
```

```
BEGIN
```

```
RAISERROR('El tratamiento ya fue aplicado a esta colmena', 16, 1);
```

```
END
```

```
END;
```

5. Función escalar para obtener la cantidad de colmenas de un apiario

```
CREATE FUNCTION fn_CantidadColmenasApiario (@ID_Apiario INT)
```

```
RETURNS INT
```

```
AS
```

```
BEGIN
```

```
DECLARE @Cantidad INT;
```

```
SELECT @Cantidad = Cant_Colmenas
```

```
FROM Apiarios
```

```
WHERE ID_Apiario = @ID_Apiario;
```

```
RETURN ISNULL(@Cantidad, 0);
```

```
END;
```

6. Función escalar para obtener la producción total de un apiario (en Kg)

```
CREATE FUNCTION fn_ProduccionTotalApiario (@ID_Apiario INT)
```

```
RETURNS DECIMAL(10,2)
```

```
AS
```

```
BEGIN
```

```
DECLARE @TotalKg DECIMAL(10,2);
```

```
SELECT @TotalKg = SUM(b.Cantidad_Miel)
```

```
FROM Realiza r
```

```
INNER JOIN Genera g ON r.ID_Cosecha = g.ID_Cosecha
```

```
INNER JOIN Barriles b ON g.ID_Barril = b.ID_Barril
```

```
WHERE r.ID_Apiario = @ID_Apiario;
```

```
RETURN ISNULL(@TotalKg, 0);
```

```
END;
```

7. Función de tabla en línea para obtener las colmenas de un estado específico

```
CREATE FUNCTION fn_ColmenasPorEstado (@Estado VARCHAR(50))
```

```
RETURNS TABLE
```

```
AS
```

```
RETURN
```

```
(
```

```
SELECT ID_Colmena, Cant_Bastidores, Fortaleza_Abejas
```

```
FROM Colmenas
```

```
WHERE Estado = @Estado
```

```
);
```

8. Función de tabla en línea para listar herramientas usadas en una visita

```
CREATE FUNCTION fn_HerramientasPorVisita (@ID_Visita INT)
```

```
RETURNS TABLE
```

```
AS
```

```
RETURN
```

```
(
```

```
SELECT h.ID_Herramienta, h.Nombre, h.Tipo, p.Cantidad
```

```
FROM Planifica p
```

```
INNER JOIN Herramientas h ON p.ID_Herramienta = h.ID_Herramienta
```

```
WHERE p.ID_Visita = @ID_Visita
```

```
);
```

## Disparadores (Triggers)

Se implementan triggers para automatizar reglas de negocio y mantener la integridad de las fechas y producciones.

1. Trigger para validar que una fecha de cosecha no sea futura

```
CREATE TRIGGER trg_ValidarFechaCosecha
ON Cosechas
AFTER INSERT, UPDATE
AS
BEGIN
IF EXISTS (SELECT 1 FROM inserted WHERE Fecha > GETDATE())
BEGIN
RAISERROR('La fecha de la cosecha no puede ser posterior a la fecha actual.', 16, 1);
ROLLBACK TRANSACTION;
END
END;
```

2. Trigger para evitar registrar cantidades negativas de miel en la cosecha

```
CREATE TRIGGER trg_ValidarKilosCosecha
ON CosechasColmenas
AFTER INSERT, UPDATE
AS
BEGIN
IF EXISTS (SELECT 1 FROM inserted WHERE CantidadKg < 0)
BEGIN
RAISERROR('La cantidad de Kg producidos no puede ser un número negativo.', 16, 1);
ROLLBACK TRANSACTION;
END
END;
```

3. Trigger para validar que la fecha de instalación de una colmena no sea futura

```
CREATE TRIGGER trg_ValidarFechaInstalacion
ON Colmenas
AFTER INSERT, UPDATE
AS
BEGIN
IF EXISTS (SELECT 1 FROM inserted WHERE FechaInstalacion > GETDATE())
BEGIN
RAISERROR('La fecha de instalación de la colmena no puede ser posterior a hoy.', 16, 1);
ROLLBACK TRANSACTION;
END
END;
```

4. Trigger para validar que el Estado de una Tarea pertenezca a los valores permitidos

```
CREATE TRIGGER trg_ValidarEstadoTarea
```

```
ON Tareas
```

```
AFTER INSERT, UPDATE
```

```
AS
```

```
BEGIN
```

```
IF EXISTS (
```

```
SELECT 1
```

```
FROM inserted
```

```
WHERE Estado NOT IN ('Programada', 'En Progreso', 'Completada', 'Cancelada')
```

```
)
```

```
BEGIN
```

```
RAISERROR('El estado de la tarea no es válido. Solo se permite: Programada, En  
Progreso, Completada o Cancelada.', 16, 1);
```

```
ROLLBACK TRANSACTION;
```

```
END
```

```
END;
```

```
GO
```